



AMANDA CR **UNIVERSIDADE FEDERAL DE LAVRAS** **ISTINA AVELINO;**
WHILGNER WHANCHELLOSK CRUZ MIRANDA

RELATÓRIO FINAL: LOCALIZADOR DE ESTRELAS

LAVRAS - MG

2025

RESUMO

O presente relatório documenta o desenvolvimento de um sistema de rastreamento de estrelas baseado no projeto StarTrack, conforme descrito no site gorkem.cc [1]. Esse sistema tem como objetivo permitir o rastreamento automatizado de corpos celestes utilizando um microcontrolador e sensores, facilitando a observação astronômica e para aplicações mais avançadas, aprimorando a captação de imagens astronômicas.

1. INTRODUÇÃO

O rastreamento astronômico é um campo essencial para a observação de corpos celestes, permitindo a captura precisa de imagens e a realização de medições astronômicas. Sistemas automatizados utilizam princípios de mecânica celeste, óptica e eletrônica para acompanhar a movimentação dos astros.

1.1. Movimentação dos Corpos Celestes

Os corpos celestes se movem no céu devido à rotação da Terra e ao seu movimento orbital. O rastreamento estelar exige a compensação desse movimento, o que pode ser feito com motores de passo controlados por microcontroladores.

1.1. Coordenadas Celeste

O posicionamento dos astros no céu é determinado por sistemas de coordenadas celestes. O mais utilizado é o sistema equatorial, que se baseia na projeção da esfera celeste e utiliza os seguintes parâmetros:

- **Ascensão Reta (AR):** Equivalente à longitude terrestre, medida em horas, minutos e segundos ao longo do equador celeste. Varia de 0h a 24h.
- **Declinação (DEC):** Equivalente à latitude terrestre, expressa em graus de -90° (Polo Sul Celeste) a $+90^\circ$ (Polo Norte Celeste).

Outro sistema relevante é o sistema altazimutal, que utiliza:

- **Azimuth (AZ):** Ângulo medido a partir do norte na horizontal, variando de 0° a 360° .

- **Altitude (ALT):** Ângulo de elevação do objeto acima do horizonte, variando de 0° (no horizonte) a 90° (no zênite, ponto diretamente acima do observador).

A conversão entre os sistemas equatorial e altazimutal é essencial para o rastreamento automático, pois o localizador deve ajustar sua posição em tempo real conforme a rotação da Terra.

1.1.2. Passos seguidos pelo arduino para a conversão de coordenadas:

Obtenção do Tempo Sideral Local

O tempo sideral local (LST) é essencial para converter RA em um ângulo observável na Terra. O código calcula o tempo sideral a partir da hora do RTC (relógio de tempo real) e da longitude do observador .

```
float calculateSiderealTime(int hours, int minutes, int seconds, float longitude) {
    float JD = 2451545.0 + (hours / 24.0) + (minutes / 1440.0) + (seconds / 86400.0);
    float SID = 280.46061837 + 360.98564736629 * (JD - 2451545.0) + longitude;
    return SID;
}
```

Aqui:

- O **Tempo Universal (UTC)** é usado para calcular o **Julian Date (JD)**.
- O JD é então usado para calcular o **tempo sideral local**, que depende da rotação da Terra.
- A longitude do observador é adicionada para ajustar o tempo sideral à sua posição na Terra.

2. Conversão das Coordenadas Celestes

A estrela tem coordenadas fixas no céu:

- **RA (Ascensão Reta)** → Localização da estrela ao longo do equador celeste, dada em horas (1 hora = 15°).
- **Dec (Declinação)** → Latitude da estrela no céu, medida em graus positivos (Norte) ou negativos (Sul).

O código converte RA de horas para graus:

```
void calculateStarCoordinates(float RA_hours, float Dec_deg, float siderealTime) {
    star_RA = RA_hours * 15.0; // 1h = 15 graus
    star_Dec = Dec_deg;
}
```

Isso é necessário porque os motores funcionam em graus, não em horas.

3. Cálculo do Ângulo de Movimento

Após converter as coordenadas celestes para graus, o código traduz esses valores para os motores do telescópio. Cada motor precisa girar um certo número de passos para alcançar a posição correta:

```
const int stepsPerDegreeRA = 15;
const int stepsPerDegreeDec = 15;

int stepsRA = star_RA * stepsPerDegreeRA;
int stepsDec = star_Dec * stepsPerDegreeDec;
```

Isso significa:

- O motor RA se move de acordo com a Ascensão Reta convertida em graus.
- O motor Dec se move conforme a declinação.

O código então faz os motores se moverem:

```
motorRA.setSpeed(60);
motorDec.setSpeed(60);

motorRA.step(stepsRA);
motorDec.step(stepsDec);
```

Isso ajusta o telescópio para a posição calculada da estrela.

1.1.3 Cálculos Manuais:

Passo 1: Calcular o Tempo Sideral Local (LST)

O Tempo Sideral Local (LST) é crucial porque determina a posição das estrelas no céu para um determinado momento e local. Ele é dado por:

$$LST = GST + \text{Longitude do observador (em graus)}$$

Onde:

- GST é o Tempo Sideral de Greenwich, que pode ser obtido de tabelas astronômicas ou calculado a partir do tempo universal (UTC).
- Longitude do observador é positiva para longitudes leste e negativa para longitudes oeste.

Exemplo de Cálculo

Se o GST for 10h 30min e o observador estiver a -47.929° de longitude, convertemos para horas:

$$- 47.929^\circ / 15 = - 3.195 \text{ horas}$$

$$LST = 10.5 - 3.195 = 7.305 \text{ horas} \approx 7h18min$$

Passo 2: Calcular o Ângulo Horário (HA)

O Ângulo Horário (HA) indica a posição da estrela em relação ao meridiano do observador e é calculado como:

$$HA = LST - RA$$

Se a estrela tem $RA = 5h 55min$, então:

$$HA = 7h18min - 5h55min = 1h23min$$

Convertendo para graus ($1h = 15^\circ$):

$$HA = 1.383 \times 15 = 20.75^\circ$$

Se HA for negativo, significa que a estrela ainda não cruzou o meridiano.

Passo 3: Converter para Coordenadas Horizontais (Altitude e Azimute)

Agora, convertemos Declinação (Dec) e Ângulo Horário (HA) para Altitude (Alt) e Azimute (Az). Para isso, usamos:

$$\sin(Alt) = \sin(Dec) \times \sin(\phi) + \cos(Dec) \times \cos(\phi) \times \cos(HA)$$

Onde:

- ϕ (phi) é a latitude do observador.
- Dec é a declinação da estrela.
- HA é o ângulo horário calculado.
- Alt é a altura da estrela acima do horizonte.
- Az é o ângulo medido do norte em sentido horário. [2]

1.2. Funcionamento dos componentes principais:

Giroscópio

O giroscópio é um sensor capaz de medir a velocidade angular em diferentes eixos. No contexto do StarTrack, ele pode ser utilizado para detectar mudanças na orientação do telescópio, auxiliando no ajuste de sua posição. Sensores como o MPU-6050 fornecem leituras de giroscópio e acelerômetro, permitindo correções automáticas de inclinação.



Módulo RTC (Real-Time Clock)

O módulo RTC é um relógio de tempo real utilizado para manter a precisão da hora do sistema. No rastreamento astronômico, ele fornece a informação do horário exato, essencial para calcular o tempo sideral local (LST) e converter coordenadas celestes para terrestres. O DS3231 é um dos módulos mais comuns devido à sua alta precisão.



Protoboard

A protoboard é uma placa de conexões que facilita a montagem dos circuitos eletrônicos sem a necessidade de soldagem. No projeto, ela permite a interligação dos motores de passo, microcontrolador e outros componentes, possibilitando testes e ajustes antes da montagem definitiva.



Motor de Passo

O motor de passo é um atuador eletromecânico que converte sinais elétricos em movimentos angulares precisos. Ele funciona por meio de pulsos elétricos enviados para suas bobinas,

permitindo o controle da posição exata do eixo. No projeto StarTrack, os motores de passo são usados para movimentar o laser em pequenas incrementações angulares, garantindo um rastreamento contínuo e preciso dos corpos celestes.



Arduino

O Arduino é a plataforma de microcontrolador utilizada para processar os dados dos sensores e controlar os motores de passo. Ele recebe as leituras do giroscópio, do módulo RTC e dos sensores de luz, calculando a posição desejada do telescópio e enviando comandos para os motores ajustarem sua orientação.



2. MATERIAIS E MÉTODOS

2.1. Materiais

Para a realização do projeto, foram utilizados os seguintes materiais:

Componentes Utilizados:

- Arduino Uno

- Motores de passo e drivers
- Giroscópio (MPU-6050)
- Módulo RTC (DS3231)
- Protoboard e fiação
- Estrutura mecânica para montagem
- Fonte de alimentação adequada
- Software para controle e análise dos dados.

2.2 Princípio de Funcionamento

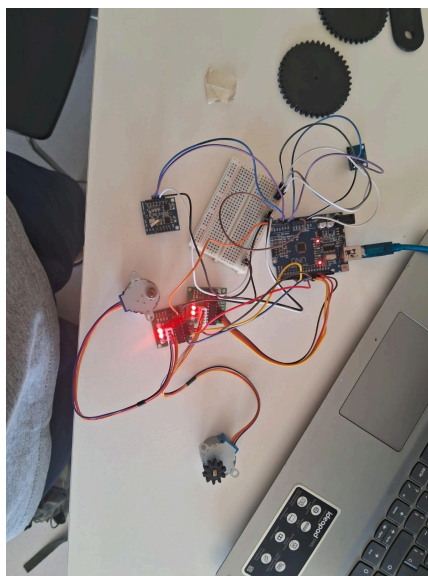
O sistema recebe os dados do módulo RTC para calcular a posição do telescópio e aciona os motores de passo para ajustá-lo automaticamente, garantindo o rastreamento da estrela-alvo em tempo real.

2.2 Procedimento Experimental

1. Montagem do circuito:

- Conectar o Arduino, os motores de passo e o módulo RTC na protoboard. [3]
- Garantir a alimentação correta dos componentes.

Fonte: Dos autores (2024)



2. Programação do Arduino:

- Implementar o código para leitura do RTC e cálculo das coordenadas estelares.
[4]
- Configurar a comunicação entre o Arduino e os motores de passo.

3. Impressão do Suporte:

- Utilizar uma impressora 3D para confeccionar as peças. [5]



Fonte: Dos autores(2025)

4. Testes iniciais:

- Verificar o funcionamento do módulo RTC e a precisão da conversão de coordenadas.
- Ajustar os parâmetros de movimento dos motores de passo.

5. Calibração do rastreamento:

- Comparar os cálculos com dados reais de posição estelar.
- Ajustar o código para corrigir possíveis imprecisões.

6. Validação do sistema:

- Testar o rastreamento de diferentes estrelas ao longo do tempo.
- Avaliar o desempenho e possíveis melhorias no sistema.

3. ORÇAMENTO

Materiais	Quantid ade	Valor estimado	Valor gasto
Motor de passo (5V)	2	R\$50,00	0
Arduino Uno	1	R\$44,00	0

Giroscópio	1	R\$20,00	0
RTC	1	R\$ 15,00	0
Rolamento de esferas 30x47x11 mm	1	R\$17,00	R\$16,80
Parafusos e porcas M3	10	R\$15,00	R\$2,00
Rolamentos 623 2rs	4	R\$20,00	R\$48,00
Jumpers	-	R\$5,00	R\$22,00
Supercola	1	R\$3,00	0
Laser	1	R\$30,00	R\$30,00
Filamentos	-	R\$50,00	0

Total estimado: R\$ 305,00

Total gasto: R\$ 118,80

4. CRONOGRAMA

Fase 1: Planejamento e Pesquisa (1º semana)

1. Definir os requisitos do projeto

Identificar componentes necessários (hardware e software).

Estudo sobre coordenadas celestes e movimentos aparentes das estrelas.

Estudo sobre integração de sensores e motores com Arduino.

1.1. Orçamento e aquisição de materiais

Listar fornecedores e adquirir componentes eletrônicos e materiais estruturais.

Prazo: 21/11/24

Fase 1: Apresentação do projeto (2º semana)

2. Apresentação

Objetivos

Justificativa e motivação

Metodologia

Prazo: 28/11/2024

Fase 2: Montagem do Hardware e Testes Básicos (3º semana)

3. Testes individuais e estudo dos componentes

Testar sensores (giroscópio/acelerômetro e GPS).
Testar motores de passo com drivers e fonte de alimentação.
Verificar a comunicação entre o Arduino e os componentes.
Entender o funcionamento de cada componente

Prazo: 05/11/2024

Fase 3: Desenvolvimento de software (4º e 5º semana)

4. Programação básica

Criar o código para ler dados dos sensores e movimentar os motores.
Implementar conversão de coordenadas celestes para locais (altitude e azimute).

Prazo: 12/12/2024

5. Integração e testes iniciais

Integrar o código de leitura de sensores com o controle dos motores.
Realizar simulações e ajustes de precisão.

Prazo: 19/12/2024

Fase 4: Montagem do Hardware (6º semana)

6. Montagem da estrutura física

Construir a base para os motores e sensores.
Fixar o Arduino e demais componentes na estrutura.

Prazo: 26/12/2024

Fase 5: Testes em campo e ajustes finais (7º e 8º semana)

7. Calibração e validação em ambiente real

Testar o dispositivo em condições reais de observação (céu noturno).
Ajustar parâmetros de precisão e movimentos.

Prazo: 02/01/2025

8. Documentação e apresentação do projeto

Preparar relatório final ou apresentação.
Realizar demonstração prática do dispositivo.

Prazo: 09/01/2025

4. ANÁLISE E DISCUSSÃO

Não foi possível verificar a precisão do rastreamento, porém, durante os testes foi observado o funcionamento correto de cada componente e foi evidenciado como pequenos desalinhamentos podem comprometer a exatidão da observação. Alguns problemas foram notados durante o processo:

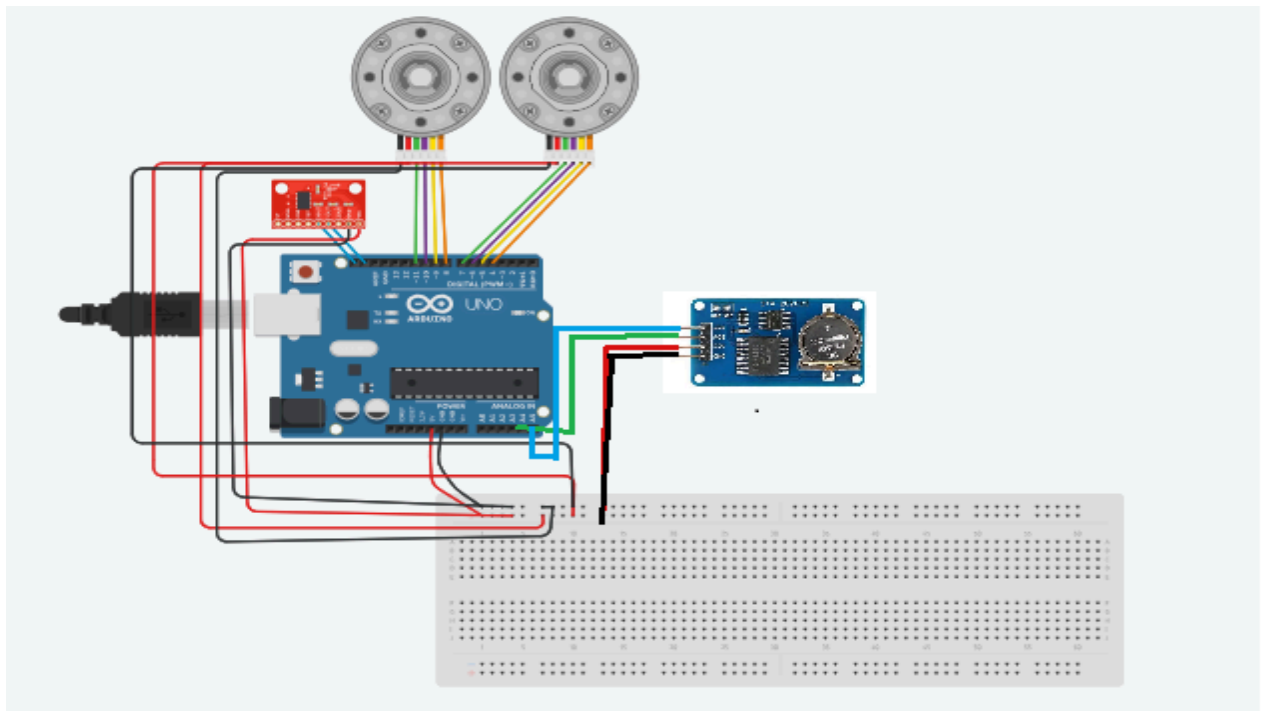
- Dificuldade no código: Houveram algumas complicações em ligar todos os componentes no arduino Uno por que alguns precisavam das mesmas portas e no momento de gerar o código, o arduino se confundia.
- Protoboard: Foi necessário o uso de uma protoboard para estender as portas de 5v e GND para que todos os componentes pudessem ser ligados corretamente.
- Impressão do suporte: Durante a impressão houve alguns problemas em algumas peças, por serem mais finas e altas precisaram de um suporte.
- Motores de passo: Houveram algumas complicações para fazer ambos funcionarem, ora um funcionava e o outro não e ora ele vibrava mas não rotacionava. Provavelmente será necessário corrigir o código ou verificar as ligações.

5. CONCLUSÃO

O projeto **StarTrack** demonstrou ser uma solução viável para observação astronômica acessível, combinando hardware simples com software inteligente. Melhorias futuras podem incluir o uso de sensores mais precisos e algoritmos de aprendizado de máquina para otimização do rastreamento.

6. REFERÊNCIAS

- [1] <https://gorkem.cc/projects/StarTrack/#step-10%3A-final-assembly-%26-tracking> - Projeto Original
- [2] https://en.wikipedia.org/wiki/Equatorial_coordinate_system - Explica os sistemas de coordenadas e suas transformações.
- [3] Esquema circuito Tinkercad - <https://www.tinkercad.com/things/2tohoZtuW3H/editel?returnTo=%2Fdashboard>



Conexões:

Motor de passo - cima/baixo

N1 → 4

N2 → 5

N3 → 6

N4 → 7

Motor de passo - direita/esquerda

N1 → 8

N2 → 9

N3 → 10

N4 → 11

Giroscópio

VCC → VCC

GND → GND

SCL → SCL

SDA → SDA

Módulo RTC

VCC → VCC

GND → GND

SCL → A5

SDA → A4

[4] Código

```
#include <Wire.h>

#include <DS3231.h>

#include <Stepper.h>

// Definir pinos para os motores de passo

const int stepsPerRevolution = 200; // Número de passos por revolução

const int motorPinRA1 = 8;

const int motorPinRA2 = 9;

const int motorPinRA3 = 10;

const int motorPinRA4 = 11;

const int motorPinDec1 = 4;

const int motorPinDec2 = 5;
```

```

const int motorPinDec3 = 6;

const int motorPinDec4 = 7;

// Criar objetos para os motores de passo

Stepper  motorRA(stepsPerRevolution,  motorPinRA1,  motorPinRA3,
motorPinRA2, motorPinRA4);

Stepper  motorDec(stepsPerRevolution,  motorPinDec1,  motorPinDec3,
motorPinDec2, motorPinDec4);

// Inicialização do RTC

DS3231 rtc;

float observer_longitude = -47.929;

float star_RA = 0.0, star_Dec = 0.0;

// Função para calcular a hora sideral (simplificação para exemplo)

float calculateSiderealTime(int hours, int minutes, int seconds, float
longitude) {

    float JD = 2451545.0 + (hours / 24.0) + (minutes / 1440.0) + (seconds
/ 86400.0);

    float SID = 280.46061837 + 360.98564736629 * (JD - 2451545.0) +
longitude;

    return SID;

}

// Função para calcular as coordenadas da estrela

void calculateStarCoordinates(float RA_hours, float Dec_deg, float
siderealTime) {

    star_RA = RA_hours * 15.0;  // Converter RA de horas para graus (1
hora = 15 graus)

```

```
    star_Dec = Dec_deg;
}

// Função para mover cada motor separadamente
void moveToStar() {
    const int stepsPerDegreeRA = 15; // Ajustado para garantir movimento perceptível
    const int stepsPerDegreeDec = 15;

    int stepsRA = star_RA * stepsPerDegreeRA;
    int stepsDec = star_Dec * stepsPerDegreeDec;

    motorRA.setSpeed(60);
    motorDec.setSpeed(60);

    Serial.print("Movendo motor RA: ");
    Serial.println(stepsRA);
    motorRA.step(stepsRA);

    delay(500); // Pequena pausa entre os motores

    Serial.print("Movendo motor Dec: ");
    Serial.println(stepsDec);
    motorDec.step(stepsDec);
}

// Função de teste para os motores (executa separadamente)
```

```
void testMotors() {

    Serial.println("Testando motor RA...");

    motorRA.setSpeed(100);

    motorRA.step(50);  // Movimento curto para teste

    delay(1000);

    Serial.println("Testando motor Dec...");

    motorDec.setSpeed(100);

    motorDec.step(50);

    delay(1000);

}

void setup() {

    Serial.begin(9600);

    // Teste inicial dos motores

    testMotors();

    // Defina as coordenadas da estrela (RA em horas, Dec em graus)

    float star_RA_hours = 5 + (55 / 60.0);

    float star_Dec_deg = 7,407;

    bool h12;

    bool PM_time;

    int hours = rtc.getHour(h12, PM_time);

    int minutes = rtc.getMinute();

    int seconds = rtc.getSecond();

}
```

```

    float siderealTime = calculateSiderealTime(hours, minutes, seconds,
observer_longitude);

    calculateStarCoordinates(star_RA_hours, star_Dec_deg, siderealTime);

    Serial.print("Coordenadas da Estrela: ");

    Serial.print("RA: ");

    Serial.print(star_RA);

    Serial.print(" graus, Dec: ");

    Serial.println(star_Dec);

    moveToStar();
}

void loop() {

    delay(5000);

    bool h12;

    bool PM_time;

    int hours = rtc.getHour(h12, PM_time);

    int minutes = rtc.getMinute();

    int seconds = rtc.getSecond();

    float siderealTime = calculateSiderealTime(hours, minutes, seconds,
observer_longitude);

    calculateStarCoordinates(6 + (46 / 60.0), -16.7161, siderealTime);

```

```
moveToStar();  
}
```

[5] Arquivo de impressão: Disponível em
<https://gorkem.cc/projects/StarTrack/#step-10%3A-final-assembly-%26-tracking>